Attributes

The v-bind directive can replace the HTML attributes.

- `<div v-bind:id="dynamicId"></div>`

For boolean attributes, v-bind does not work the same where the mere existence implies true. For example:

- `<button v-bind:disabled="disabled">Button</button>`

If the value for disabled is false, undefined or null, the <button> element will not be rendered and include the disabled attribute.

Using JavaScript Expressions

The text that we put in the curly brackets is known as binding expressions. A binding expression, in Vue.js, comprises a standalone JavaScript expression followed by one or more filters.

In our templates, we usually bind simple property keys. However, vue.js supports all data bindings lying inside to the best of JavaScript expressions.

- `{{ value + 1 }}`
- `{{satisfied ? 'YES' : 'NO' }}`
- `{{sentence.split('').reverse().join('')}}`
- `<ul v-bind:id="'product-'+id"></ul>`

These expressions will be checked as JavaScript while deciding the data scope of the owner Vue instance. But it is worth noting that this binding cannot have more than one single expression. It will not work if there is more than one expression.

Filters

In Vue.js, you can use filters at the end of the expression. It is marked as the "pipe" symbol. Here is an example:

- `{{ message | capitalize }}`

The pipe symbol means we are piping the message expression value with the help of a built-in "capitalize" filter. However, do not include the pipe syntax inside the expressions. They are only to be used outside and not make it a part of the JavaScript expression.

Directives

The special attributes that come with the v- prefix, directives, and directive attribute values are usually considered to render as a single JavaScript expression. There is just one exception, and that is for v-for. Whenever there is a change in expression values, you can apply side effects to Document